

1. Introduction

The Testing & Validation Report documents the verification process performed to ensure the successful deployment and operation of the AI-Powered SOC Assistant integrated with Wazuh SIEM. The report validates the functionality of each system component, confirms end-to-end communication between modules, and demonstrates that AI-based alert enrichment operates as expected.

2. Objectives

The objectives of testing were:

- Verify Docker installation and container health.
 - Validate Wazuh deployment.
 - Confirm Windows Agent connectivity.
 - Ensure alerts are generated and received.
 - Verify AI Monitor detects new alerts.
 - Validate AI Analyzer enriches alerts.
 - Test continuous monitoring functionality.
 - Confirm overall end-to-end integration.
-

3. Test Environment

Component	Details
Host OS	Kali Linux
Endpoint	Windows 11
SIEM	Wazuh 4.12
Container Platform	Docker
Programming Language	Python 3.13
Dashboard	Wazuh Dashboard
Data Format	JSON

4. Testing Strategy

Testing was conducted in four stages:

Stage 1 – Infrastructure Validation

Verified Docker installation and Wazuh deployment.

Stage 2 – Agent Validation

Verified Windows Agent registration and communication.

Stage 3 – AI Validation

Verified AI Monitor and AI Analyzer functionality.

Stage 4 – End-to-End Validation

Verified the complete flow from Windows event generation to AI-enriched output.

5. Test Cases

Test Case TC-01

Objective

Verify Docker installation.

Test Steps

1. Install Docker.
2. Start Docker service.
3. Execute:

```
docker ps
```

Expected Result

Docker starts successfully and lists running containers.

Actual Result

Docker started successfully and displayed active containers.

Status

 Pass

Test Case TC-02

Objective

Deploy Wazuh Docker Stack.

Test Steps

Run:

```
docker compose up -d
```

Expected Result

Manager, Indexer, and Dashboard containers are running.

Actual Result

All required containers started successfully.

Status

 Pass

Test Case TC-03

Objective

Verify Dashboard Access.

Test Steps

Open:

```
https://<Manager-IP>
```

Expected Result

Dashboard loads successfully.

Actual Result

Dashboard opened and administrator login was successful.

Status

 Pass

Test Case TC-04

Objective

Verify Windows Agent Registration.

Test Steps

Authenticate using:

agent-auth.exe -m <Manager-IP>

Expected Result

Agent receives authentication key.

Actual Result

Agent registered successfully.

Status

 Pass

Test Case TC-05

Objective

Verify Agent Connectivity.

Test Steps

Navigate to:

Dashboard → Agents

Expected Result

Agent status is **Active**.

Actual Result

Windows endpoint displayed as Active.

Status

 Pass

Test Case TC-06

Objective

Generate Windows Security Event.

Test Steps

Perform a Windows login.

Expected Result

Wazuh detects the authentication event.

Actual Result

Windows Logon Success alert generated.

Status

 Pass

Test Case TC-07

Objective

Verify alerts.json Updates.

Test Steps

Inspect the latest alert inside the Wazuh Manager container.

Expected Result

A new JSON alert is written.

Actual Result

Latest alert successfully retrieved.

Status

 Pass

Test Case TC-08

Objective

Verify AI Monitor.

Test Steps

Run:

```
python ai_monitor.py
```

Expected Result

Latest alert is detected and displayed.

Actual Result

AI Monitor displayed the latest Wazuh alert.

Status

 Pass

Test Case TC-09

Objective

Validate AI Analysis.

Test Steps

Allow AI Analyzer to process the alert.

Expected Result

AI generates:

- Threat Category
- Risk Level
- MITRE ATT&CK Mapping
- Executive Summary
- Recommendations
- Confidence Score

Actual Result

All expected analysis fields were generated successfully.

Status

 Pass

Test Case TC-10

Objective

Validate Continuous Monitoring.

Test Steps

Generate multiple Windows events while the AI Monitor is running.

Expected Result

New alerts are processed without restarting the application.

Actual Result

AI Monitor continuously detected and analyzed new alerts.

Status

 Pass

6. Integration Testing

The following integration points were verified:

Integration	Result
Windows → Wazuh Agent	 Successful
Agent → Wazuh Manager	 Successful
Manager → alerts.json	 Successful
alerts.json → AI Monitor	 Successful
AI Monitor → AI Analyzer	 Successful
AI Analyzer → SOC Output	 Successful

7. Performance Testing

Metric	Observation
Alert Detection	Near Real-Time
JSON Parsing	Successful
AI Processing	Completed within acceptable time for demonstration
Continuous Monitoring	Stable during testing

8. Error Handling Validation

The following issues were encountered and resolved during development:

Issue	Resolution
Docker container startup issues	Verified Docker service and container status
Missing alerts.json path	Accessed the file inside the Wazuh Manager container
JSON parsing errors	Added handling for incomplete alert entries
Python indentation and syntax errors	Corrected formatting and code structure
OpenAI API authentication (401)	Corrected API key configuration
OpenAI quota limitation (429)	Switched to the local rule-based analyzer for continued testing

9. Challenges Encountered

During implementation, several practical challenges were identified:

- Docker deployment and container initialization.
- SSL certificate generation for Wazuh.
- Windows Agent registration and authentication.
- Locating alerts.json inside the Docker container.
- JSON decoding errors caused by partially written log entries.
- Python syntax and indentation issues.
- API authentication and quota limitations during OpenAI integration.
- Ensuring continuous monitoring without repeatedly processing the same alert.

These challenges were investigated and resolved through configuration updates, code improvements, and validation testing.

10. Test Results Summary

Test Area	Result
Docker Deployment	✅ Pass

Test Area	Result
Wazuh Deployment	✓ Pass
Dashboard Access	✓ Pass
Agent Registration	✓ Pass
Agent Connectivity	✓ Pass
Alert Generation	✓ Pass
AI Monitor	✓ Pass
AI Analysis	✓ Pass
Continuous Monitoring	✓ Pass
End-to-End Integration	✓ Pass

11. Conclusion

Testing confirmed that the AI-Powered SOC Assistant successfully integrates with Wazuh SIEM and provides automated enrichment of Windows security alerts. The solution demonstrated reliable communication between the Windows endpoint, Wazuh components, and the AI modules. All planned validation scenarios were completed successfully, confirming that the system is suitable for demonstration and forms a solid foundation for future enhancements such as advanced AI models and external threat intelligence integration.